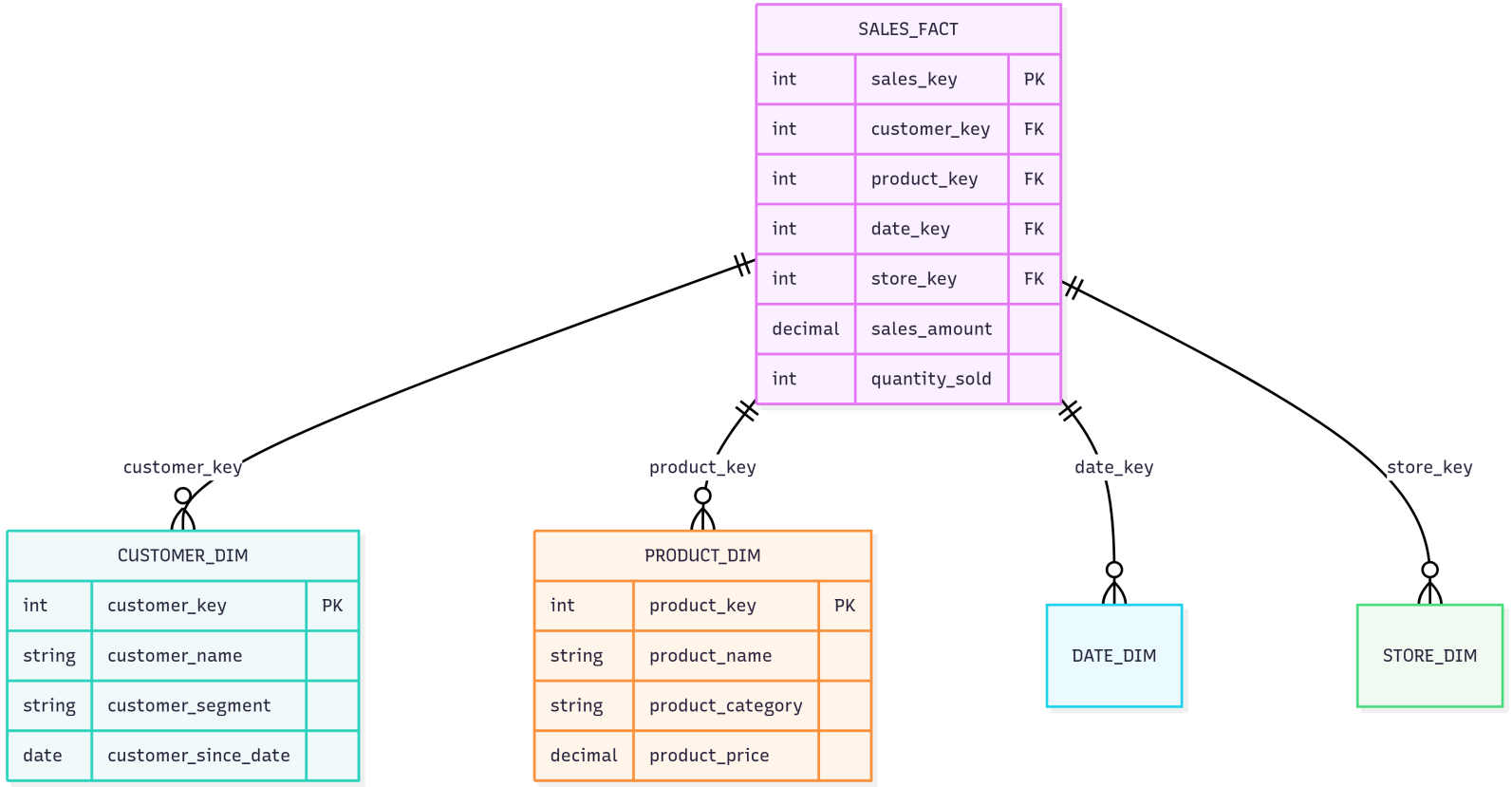
Analytics Engineering: Session 4

Data Modeling & Modularity

Agenda

- Dimensional Modeling (Star Schema)
- The DRY Principle
- dbt Project Structure: Staging, Intermediate, Marts
- Python Models in dbt
- The DAG
- Test as You Build

Dimensional Modeling



Dimensional Modeling

The **Star Schema** organizes data into two types:

- **Dimensions** (`dim_`): Descriptive attributes. *Who, What, Where, When.*
  - Example: `dim_customers`, `dim_products`
- **Facts** (`mart_`): Measurable events. *How much, How many.*
  - Example: `mart_orders`, `mart_revenue_by_segment`

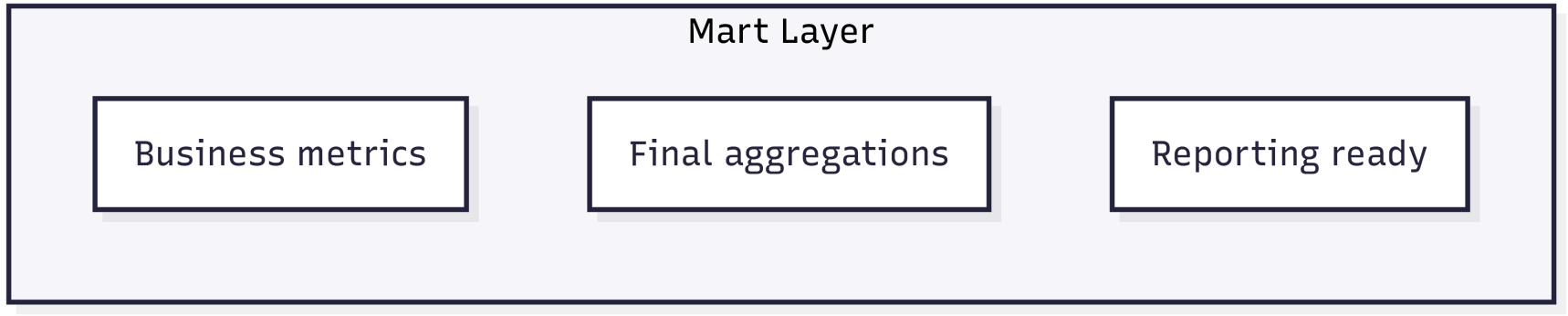
Facts are large (millions of rows). Dimensions are small (thousands of rows).

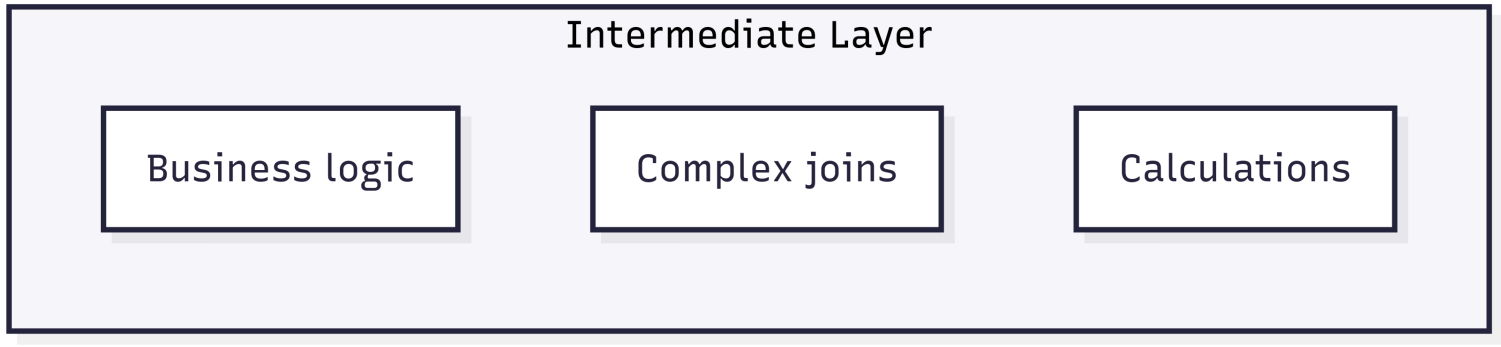
The DRY Principle

"Don't Repeat Yourself"

- If you write the same SQL logic twice, refactor it.
- **Modularity**: Build a model once, reference it (`{{ ref() }}`) many times.
- **Benefits**:
  - **Maintainability**: Fix a bug in one place, it fixes everywhere.
  - **Consistency**: Everyone uses the same definition of "Revenue".

dbt Project Structure (The Layers)





dbt Project Structure (The Layers)

We don't just jump from Source to Star Schema. We build in layers to manage complexity.

1. **Staging**: Clean and Standardize (1:1 with Source)
2. **Intermediate**: Logic and Joins (Internal).
3. **Marts**: Business Ready (Facts & Dimensions).

Layer 1: Staging

Goal: Create a reliable foundation.

- **Source-Concentric**: One model per source table.
- **Tasks**:
  - Renaming (`cust_id` → `customer_id`).
  - Casting types (`string` → `date`).
  - Basic cleaning (null handling).
- **Anti-patterns**: No Joins, No Aggregations.

Layer 2: Intermediate

Goal: Handle complexity and prepare for Marts.

- **Logic-Concentric**: Bridges the gap between Staging and Marts.
- **Tasks**:
  - Joins (e.g., `orders` + `order_items`).
  - Complex calculations.
  - Pivoting.
- **Internal**: Not exposed to end users.

Layer 3: Marts

Goal: Business Consumption.

- **Business-Concentric**: The final Star Schema.
- **Structure**:
  - `dim_xxx`: Dimension tables. Used for context and attributes, for example, customers, products.
  - `mart_xxx`: Fact/aggregate tables. Used for measurements and metrics. For example, orders, revenue.
- **Exposed**: This is what BI tools (Tableau, Looker) connect to, and what analysts query.

Python Models in dbt

dbt supports Python models for transformations hard to do in SQL (e.g., complex parsing, ML, API calls).

```python
def model(dbt, session):
    dbt.config(materialized="table")

    df = dbt.ref("my_sql_model")
    # Use Polars or Pandas logic
    df = df.filter(df["amount"] > 100)

    return df
```

We will explore Python models more in future sessions, here's the docs

Note: Requires a data platform that supports Python (DuckDB, Snowflake, BigQuery).

The DAG (Directed Acyclic Graph)

Visualizes the flow of data through your project.

- **Directed**: Data flows one way (Source → Mart).
- **Acyclic**: No loops allowed.
- **Graph**: Nodes (models) and Edges (dependencies).

A clean DAG flows left-to-right: Sources → Staging → Intermediate → Marts.

Example DAG

Let's create a simple DAG for our ecommerce dataset:

`stg_customers` → `int_customers` → `dim_customers`

Use the seed `segments.csv` to enrich the data.

Test as You Build

As you create new models, add generic tests immediately:

```yaml
models:
  - name: dim_customers
    columns:
      - name: customer_id
        tests:
          - unique
          - not_null
```

Run `dbt build` to build models **and** run their tests in DAG order.

Don't wait until the end — testing from the start catches issues early.

What have we learned in this session

- Dimensional Modeling basics (Facts vs Dims).
- The importance of DRY and Modularity.
- The 3 Layers: Staging, Intermediate, Marts.
- How Python models fit in.
- The structure of a clean DAG.
- Adding tests as you build new models.

Next Session: Practice Session I: Building the Foundation.